

Homework 4: STLC

The appendix gives the formalization of STLC and its extensions with products and sums.

1. For each of the types below, write a closed term having that type. Give a complete typing derivation for your term. You can find the typing rules for product and sum types in the appendix.

(a) $(\tau_1 \rightarrow \tau_2) \rightarrow \tau_1 \rightarrow (\tau_2 + \tau_3)$

(b) $(\tau_1 \times \tau_2) + (\tau_3 \times \tau_4) \rightarrow (\tau_1 + \tau_4)$

2. Option types.

In this problem we add option types to STLC. **None** represents no element, and **Some** M contains an element M . **get** M extracts the element from M .

Syntax:

$$\begin{array}{ll} \text{(Types)} & \tau ::= \dots \mid \text{option } \tau \\ \text{(Terms)} & M ::= \dots \mid \text{None} \mid \text{Some } M \mid \text{get } M \\ \text{(Values)} & v ::= \dots \mid \text{None} \mid \text{Some } v \end{array}$$

New reduction rules:

$$\frac{M \longrightarrow M'}{\text{Some } M \longrightarrow \text{Some } M'} \text{ (SOME)}$$

$$\frac{M \longrightarrow M'}{\text{get } M \longrightarrow \text{get } M'} \text{ (GET-M)}$$

$$\frac{}{\text{get } (\text{Some } M) \longrightarrow M} \text{ (GET-SOME)}$$

$$\frac{}{\text{get None} \longrightarrow \text{get None}} \text{ (GET-NONE)}$$

- (a) Give three appropriate new typing rules, one for each new form of term. Your rules should ensure preservation and progress. No proofs are required.
- (b) **Preservation and progress after changing rules.** Each variant changes the *original* language independently. All other rules and values remain unchanged.

Variant 1. Remove SOME.

Variant 2. Add

$$\frac{}{\text{get } v \longrightarrow \text{get } v} \text{ (GET-V)}$$

Variant 3. Replace GET-SOME by

$$\frac{}{\text{get (Some } v) \longrightarrow v} \text{ (GET-SOME')}$$

Variant 4. Replace GET-NONE by

$$\frac{}{\text{get None} \longrightarrow \text{None}} \text{ (GET-NONE')}$$

Variant 5. Remove GET-NONE and add `get None` to the syntax of values.

Variant 6. Replace the typing rule for “get” by

$$\frac{\Gamma \vdash M : \tau}{\Gamma \vdash \text{get } M : \tau}$$

Fill the table with *holds* or *fails*, using preservation and progress as stated in the appendix. For each failed property, give a counterexample and explain why it violates that property. No explanation is required for a *holds* entry.

Variant	Preservation	Progress
1		
2		
3		
4		
5		
6		

Appendix

A The Simply-Typed λ -Calculus

Syntax (here T denotes the base type):

$$\begin{aligned} (\text{Types}) \quad \tau &::= \mathsf{T} \mid \tau \rightarrow \tau \\ (\text{Terms}) \quad M &::= x \mid \lambda x : \tau. M \mid M N \\ (\text{Values}) \quad v &::= \lambda x : \tau. M \\ (\text{Contexts}) \quad \Gamma &::= \bullet \mid \Gamma, x : \tau \end{aligned}$$

Here Γ is a finite mapping from variable names to types. In a context extension $\Gamma, x : \tau$, we require $x \notin \text{dom}(\Gamma)$. Typing is considered up to α -equivalence, so bound variables may be renamed to satisfy this condition.

Free variables:

$$\begin{aligned} fv(x) &\stackrel{\text{def}}{=} \{x\}, \\ fv(M N) &\stackrel{\text{def}}{=} fv(M) \cup fv(N), \\ fv(\lambda x : \tau. M) &\stackrel{\text{def}}{=} fv(M) - \{x\}. \end{aligned}$$

Substitution (here x and y are distinct variable names):

$$\begin{aligned} x[N/x] &\stackrel{\text{def}}{=} N, \\ y[N/x] &\stackrel{\text{def}}{=} y, \\ (M_1 M_2)[N/x] &\stackrel{\text{def}}{=} (M_1[N/x]) (M_2[N/x]), \\ (\lambda x : \tau. M)[N/x] &\stackrel{\text{def}}{=} \lambda x : \tau. M, \\ (\lambda y : \tau. M)[N/x] &\stackrel{\text{def}}{=} \lambda y : \tau. (M[N/x]) \quad \text{if } y \notin fv(N), \\ (\lambda y : \tau. M)[N/x] &\stackrel{\text{def}}{=} \lambda z : \tau. (M[z/y])[N/x] \quad \text{if } y \in fv(N) \text{ and } z \text{ fresh.} \end{aligned}$$

Reduction rules:

$$\begin{aligned} \frac{}{(\lambda x : \tau. M) N \longrightarrow M[N/x]} \quad & \frac{M \longrightarrow M'}{\lambda x : \tau. M \longrightarrow \lambda x : \tau. M'} \\ \frac{M \longrightarrow M'}{M N \longrightarrow M' N} \quad & \frac{N \longrightarrow N'}{M N \longrightarrow M N'} \end{aligned}$$

Multistep reduction ($k \geq 0$):

$$\begin{aligned} M \longrightarrow^0 M' &\quad \text{iff } M = M', \\ M \longrightarrow^{k+1} M' &\quad \text{iff } \exists M''. M \longrightarrow M'' \wedge M'' \longrightarrow^k M', \\ M \longrightarrow^* M' &\quad \text{iff } \exists k \geq 0. M \longrightarrow^k M'. \end{aligned}$$

Normal form: a term M for which no M' satisfies $M \longrightarrow M'$.

Closed term: a term M with $fv(M) = \emptyset$.

Typing rules:

$$\begin{array}{c} \frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau} \text{ (VAR)} \qquad \frac{\Gamma, x : \tau \vdash M : \tau'}{\Gamma \vdash (\lambda x : \tau. M) : \tau \rightarrow \tau'} \text{ (ABS)} \\[10pt] \frac{\Gamma \vdash M : \tau \rightarrow \tau' \quad \Gamma \vdash N : \tau}{\Gamma \vdash M N : \tau'} \text{ (APP)} \end{array}$$

Preservation:

For any M, M' and τ , if $\bullet \vdash M : \tau$ and $M \longrightarrow M'$, then $\bullet \vdash M' : \tau$.

Progress:

For any M and τ , if $\bullet \vdash M : \tau$, then either $M \in \text{Values}$ or $\exists M'. M \longrightarrow M'$.

A.1 Products

Syntax:

$$\begin{array}{ll} \text{(Types)} & \tau ::= \dots \mid \tau \times \tau \\ \text{(Terms)} & M ::= \dots \mid \langle M, M \rangle \mid \text{proj1 } M \mid \text{proj2 } M \\ \text{(Values)} & v ::= \dots \mid \langle v, v \rangle \end{array}$$

Free variables:

$$\begin{aligned} fv(\langle M, N \rangle) &\stackrel{\text{def}}{=} fv(M) \cup fv(N), \\ fv(\text{proj1 } M) &\stackrel{\text{def}}{=} fv(M), \quad fv(\text{proj2 } M) \stackrel{\text{def}}{=} fv(M). \end{aligned}$$

Typing rules:

$$\begin{array}{c} \frac{\Gamma \vdash M : \sigma \quad \Gamma \vdash N : \tau}{\Gamma \vdash \langle M, N \rangle : \sigma \times \tau} \text{ (PAIR)} \\[10pt] \frac{\Gamma \vdash M : \sigma \times \tau}{\Gamma \vdash \text{proj1 } M : \sigma} \text{ (PROJ1)} \quad \frac{\Gamma \vdash M : \sigma \times \tau}{\Gamma \vdash \text{proj2 } M : \tau} \text{ (PROJ2)} \end{array}$$

Reduction rules:

$$\begin{array}{c} \overline{\text{proj1 } \langle M, N \rangle \longrightarrow M} \quad \overline{\text{proj2 } \langle M, N \rangle \longrightarrow N} \\[10pt] \frac{M \longrightarrow M'}{\langle M, N \rangle \longrightarrow \langle M', N \rangle} \quad \frac{N \longrightarrow N'}{\langle M, N \rangle \longrightarrow \langle M, N' \rangle} \\[10pt] \frac{M \longrightarrow M'}{\text{proj1 } M \longrightarrow \text{proj1 } M'} \quad \frac{M \longrightarrow M'}{\text{proj2 } M \longrightarrow \text{proj2 } M'} \end{array}$$

A.2 Sums

Syntax:

$$\begin{array}{ll}
\text{(Types)} & \tau ::= \dots \mid \tau + \tau \\
\text{(Terms)} & M ::= \dots \mid \text{left } M \mid \text{right } M \mid \text{case } M \text{ do } M_1 M_2 \\
\text{(Values)} & v ::= \dots \mid \text{left } v \mid \text{right } v
\end{array}$$

Free variables:

$$\begin{aligned}
fv(\text{left } M) &\stackrel{\text{def}}{=} fv(M), & fv(\text{right } M) &\stackrel{\text{def}}{=} fv(M), \\
fv(\text{case } M \text{ do } M_1 M_2) &\stackrel{\text{def}}{=} fv(M) \cup fv(M_1) \cup fv(M_2).
\end{aligned}$$

Typing rules:

$$\begin{array}{c}
\frac{\Gamma \vdash M : \sigma}{\Gamma \vdash \text{left } M : \sigma + \tau} \text{ (LEFT)} \quad \frac{\Gamma \vdash M : \tau}{\Gamma \vdash \text{right } M : \sigma + \tau} \text{ (RIGHT)} \\
\\
\frac{\Gamma \vdash M : \sigma + \tau \quad \Gamma \vdash M_1 : \sigma \rightarrow \rho \quad \Gamma \vdash M_2 : \tau \rightarrow \rho}{\Gamma \vdash \text{case } M \text{ do } M_1 M_2 : \rho} \text{ (CASE)}
\end{array}$$

Reduction rules:

$$\begin{array}{c}
\frac{}{\text{case } (\text{left } M) \text{ do } M_1 M_2 \longrightarrow M_1 M} \\
\\
\frac{}{\text{case } (\text{right } M) \text{ do } M_1 M_2 \longrightarrow M_2 M} \\
\\
\frac{M \longrightarrow M'}{\text{left } M \longrightarrow \text{left } M'} \quad \frac{M \longrightarrow M'}{\text{right } M \longrightarrow \text{right } M'} \\
\\
\frac{M \longrightarrow M'}{\text{case } M \text{ do } M_1 M_2 \longrightarrow \text{case } M' \text{ do } M_1 M_2} \\
\\
\frac{M_1 \longrightarrow M'_1}{\text{case } M \text{ do } M_1 M_2 \longrightarrow \text{case } M \text{ do } M'_1 M_2} \\
\\
\frac{M_2 \longrightarrow M'_2}{\text{case } M \text{ do } M_1 M_2 \longrightarrow \text{case } M \text{ do } M_1 M'_2}
\end{array}$$

Substitution extends componentwise to products and sums.
Preservation and progress also hold with products and sums.